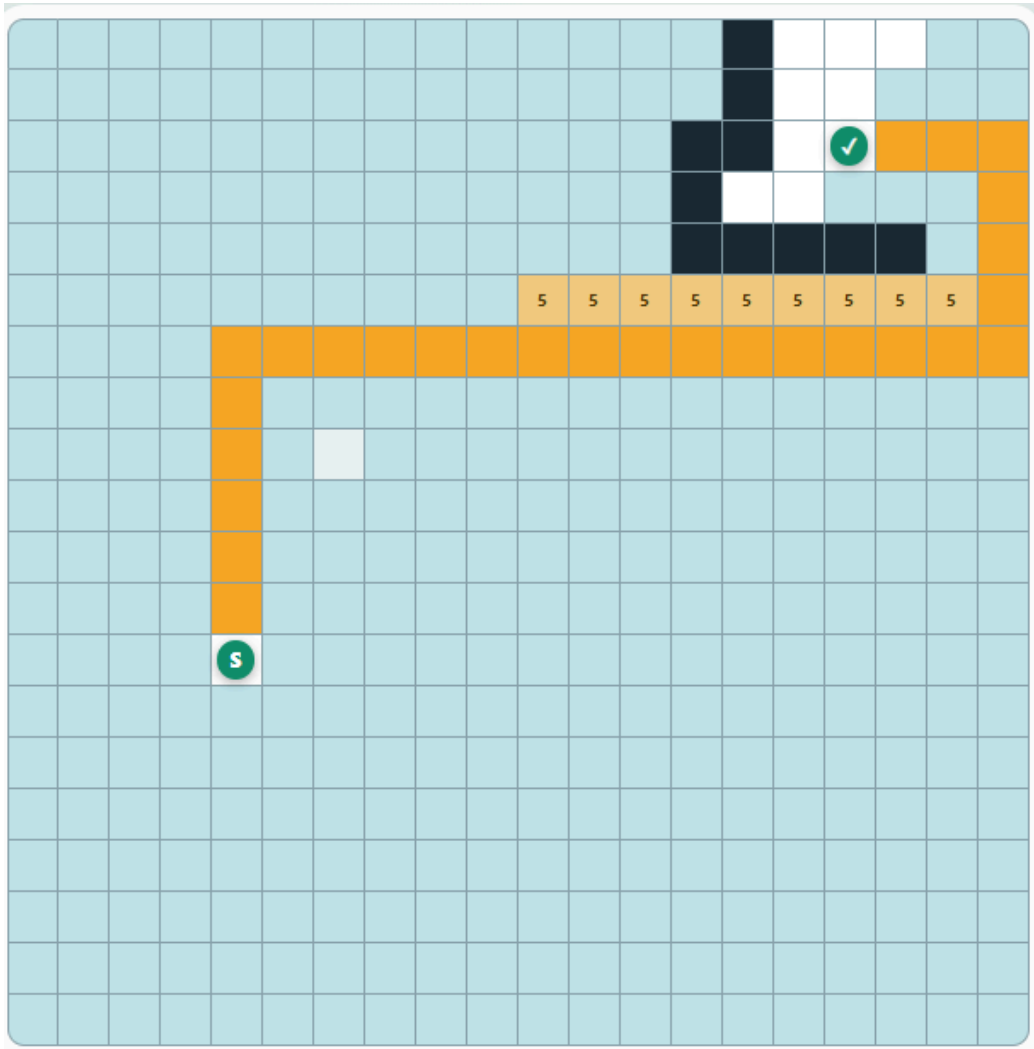


INFORME TÉCNICO: LABORATORIO DE RUTAS Y ALGORITMOS DE BÚSQUEDA



Autor: Igor Gomez

Módulo: Programación de Inteligencia Artificial

Repositorio GitHub: https://github.com/igorr17/Laberintos_Igor

Despliegue en Netlify: <https://laberintos-igor.netlify.app>

ÍNDICE

INFORME TÉCNICO: LABORATORIO DE RUTAS Y ALGORITMOS DE BÚSQUEDA..	1
ÍNDICE	2
1. Introducción y Justificación Técnica.....	3
2. Paso a paso de construcción: Decisiones y Correcciones.....	4
3. Arquitectura del Proyecto.....	5
4. Algoritmos de Búsqueda.....	6
5. Tabla Comparativa de Laberintos (Desafíos).....	7
Comparación cruzada: los cuatro algoritmos sobre un mismo tablero.....	9
6. Casos de Prueba Obligatorios.....	10
7. Registro de Uso de Asistentes IA.....	11

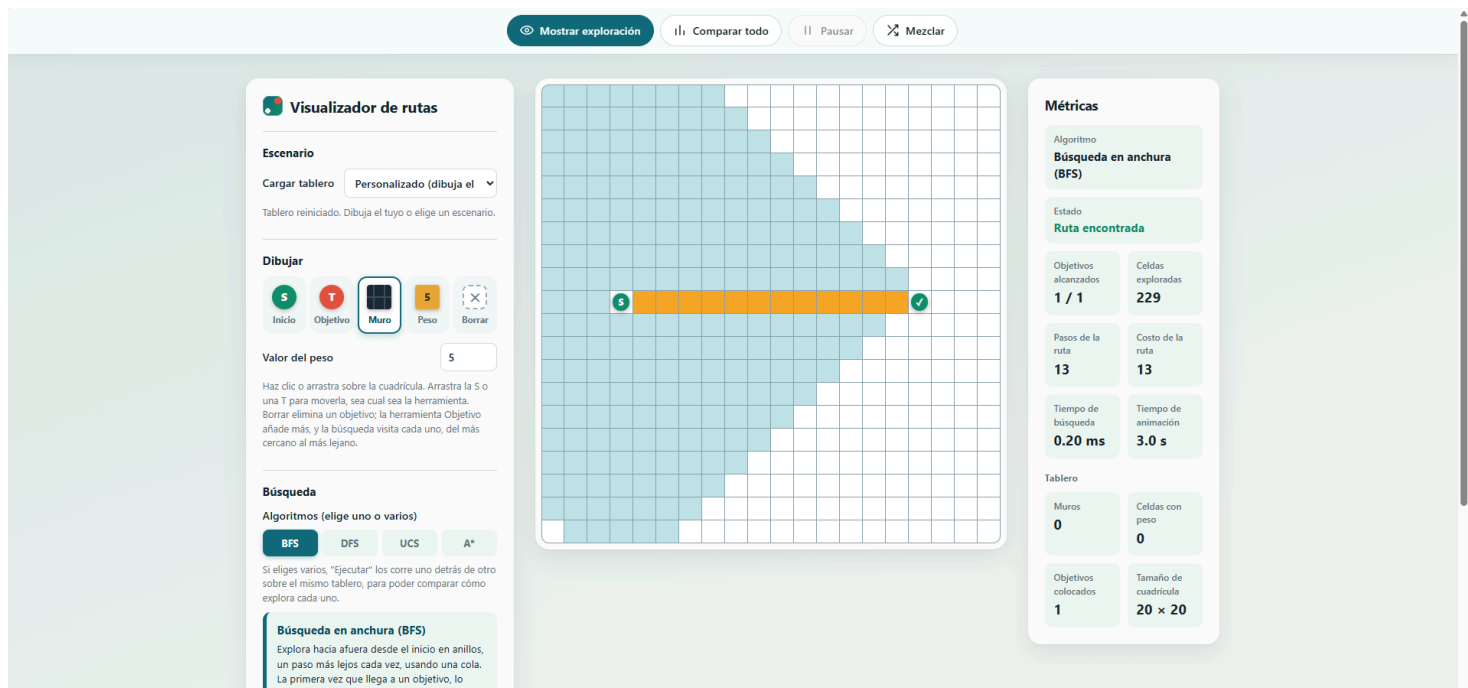
1. Introducción y Justificación Técnica

El proyecto partió de la demostración visual en la pizarra digital en clase. El objetivo planteado era reproducir de forma autónoma ese laboratorio de rutas desde cero, sin plantillas ni código base facilitado, estructurando obligatoriamente la solución en los tres archivos web fundamentales (index.html, style.css y script.js) y asegurando que funcione tanto en local como desplegado en Netlify.

- ¿Por qué JavaScript?: Porque funciona directamente en cualquier navegador sin tener que instalar programas adicionales. Además, nos permite meter pausas fáciles para ver con los ojos cómo el algoritmo va explorando casilla por casilla en la pantalla, en vez de escupirte el resultado de golpe en una consola de texto.
- ¿Por qué código web básico (sin librerías ni frameworks)?: Porque para un tablero de 20x20 meter cosas complejas solo añade peso innecesario. Con JavaScript pelado la página carga al instante, no da fallos de dependencias y va súper fluida cuando haces clic o arrastras el ratón para pintar muros.
- ¿Por qué va perfecto para Netlify?: Al ser solo tres archivos de texto normales (HTML, CSS y JS), no hace falta compilar nada ni pelearse con configuraciones de servidor. Se suben tal cual a Netlify y la aplicación ya funciona para que cualquiera la abra con un enlace.

2. Paso a paso de construcción: Decisiones y Correcciones

- **Planteamiento inicial con la IA:** Le pedí a Claude que generará la estructura básica dividida en los tres archivos (HTML, CSS y JS). Para el tablero de 20x20, la IA propuso no usar una matriz de dos dimensiones [fila][columna], sino un array plano de 400 posiciones donde cada celda se calcula con $(\text{fila} * 20 + \text{columna})$. Al revisarlo entendí la ventaja: encontrar los vecinos (arriba, abajo, izquierda, derecha) es mucho más directo simplemente sumando o restando posiciones en el array.
- **Optimización de clics:** Al generar el tablero, la IA implementó un único escuchador de eventos en el contenedor general en vez de poner 400 escuchadores individuales (uno por casilla), lo que hace que pintar muros al arrastrar el ratón vaya completamente fluido.
- **Corrección del arrastre de nodos:** Al probar la web en el navegador encontré un fallo: si tenía seleccionada la herramienta 'Muro' e intentaba arrastrar el nodo de inicio, la aplicación pintaba un muro encima del inicio en vez de moverlo. Le pedí a la IA que corrigiera esto dando prioridad siempre a mover el nodo si hacías clic sobre él, independientemente de la herramienta que estuviera activa.
- **Ajuste de cuadrícula fija:** Al redimensionar la ventana del navegador el tablero cambiaba de tamaño. Le pedí que fijara el tamaño exacto en píxeles (616x616 px) para que la cuadrícula no se deformara en ninguna pantalla.



3. Arquitectura del Proyecto

El proyecto fluye de manera sencilla pero robusta:

- **index.html:** Contiene el lenguaje de marcado. Declara semánticamente los bloques (botones, menús, cuadrícula y panel de métricas). No ejecuta lógica.
- **style.css:** Lenguaje de estilos declarativo. Usa Flexbox y CSS Grid para que el tablero nunca se deforme.
- **script.js:** El lenguaje de programación. Maneja un flujo claro: el usuario al hacer clic se actualizan los arrays de estado (walls, weights) -> la función render() dibuja en pantalla. Al ejecutar la búsqueda, se calcula toda la ruta en memoria primero y luego un bucle asíncrono la anima visualmente.

 index	01/10/2026 9:14	Microsoft Edge H...	7 KB
 script	01/10/2026 9:14	Archivo de origen ...	22 KB
 style	01/10/2026 9:14	Archivo de origen ...	14 KB

4. Algoritmos de Búsqueda

He implementado una única función central `search()` que actúa de forma distinta según el algoritmo:

- **BFS (Anchura):** Utiliza una estructura FIFO (Cola). Avanza en forma de anillo. Ideal para encontrar la ruta con menos pasos porque trata todos los costes por igual, pero ignora los pesos del terreno.
- **DFS (Profundidad):** Usa una estructura LIFO (Pila). Avanza por una rama hasta el fondo antes de retroceder. No garantiza la ruta más corta ni la más barata.
- **UCS (Coste Uniforme):** Actúa como una cola de prioridad basándose en el coste acumulado. Expande el nodo más barato. Perfecto para esquivar los pesos altos del tablero.
- **A* (A-Estrella):** Combina UCS con una heurística. Uso la "Distancia Manhattan" (diferencia de filas + diferencia de columnas hasta la meta). Esto le da un sentido de "dirección" y explora muchísimas menos celdas asegurando el coste mínimo.
- **Binary Search:** Aunque no es un algoritmo para recorrer grafos espaciales por adyacencia (ya que no podemos "partir" un laberinto con muros a la mitad como en un array ordenado), lo tengo en cuenta para futuras mejoras, por ejemplo, buscando umbrales de coste máximo.

5. Tabla Comparativa de Laberintos (Desafíos)

Se seleccionaron los cuatro escenarios del menú que ilustran, cada uno con su algoritmo recomendado, las cuatro categorías pedidas: menos pasos, pesos, objetivo con costo y sin solución. Los valores se leyeron del panel de métricas tras ejecutar cada escenario de principio a fin en la aplicación real (velocidad de animación al máximo, sin alterar el resultado del algoritmo, que se calcula antes de animarse).

Escenario	Algoritmo	Pasos	Costo	Exploradas	Conclusión personal
1. Menos pasos (Ruta limpia)	BFS	23	23	344	Encuentra el camino recto más rápido por la abertura, ideal sin pesos.
2. Atajo caro (Desvío)	UCS	23	23	341	UCS prefiere dar un pequeño rodeo antes que atravesar baldosas muy caras, logrando el coste mínimo.
3. Varias metas	A*	45	45	62	Prioriza por coste y cercanía. La heurística reduce drásticamente las celdas exploradas (solo 62) frente a las demás búsquedas.
4. Sin solución	DFS / Todos	0	0	375	Exploran todo lo accesible y detectan correctamente que no hay solución sin colgar la app.



Fig. 4.1 — Escenario 1, BFS: ruta de 23 pasos por la abertura más cercana.



Fig. 4.2 — Escenario 2, UCS: rodeo del bloque de peso 9.

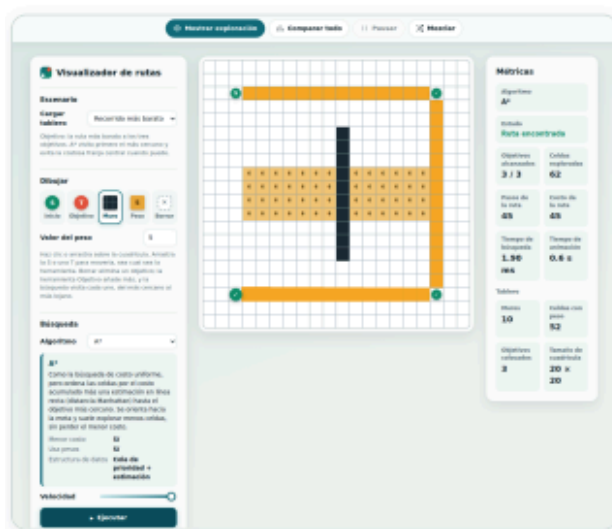


Fig. 4.3 — Escenario 3, A*: tres objetivos, evitando la franja de peso 6.

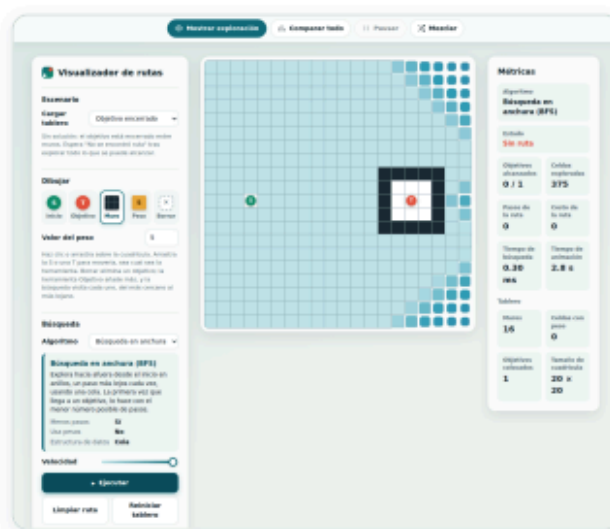


Fig. 4.4 — Escenario 4: objetivo encerrado, sin solución.

Comparación cruzada: los cuatro algoritmos sobre un mismo tablero

Como evidencia adicional de las diferencias entre algoritmos, se usó el botón "Comparar todo" sobre el tablero del escenario 3 (tres objetivos, pesos y muros), que ejecuta BFS, DFS, UCS y A* sobre el mismo tablero sin animación y cronometra cada uno.

Algoritmo	Objetivos	Exploradas	Pasos	Costo	Tiempo
BFS	3 / 3	547	45	45	0.30 ms
DFS	3 / 3	466	439	699	2.40 ms
UCS	3 / 3	492	45	45	0.50 ms
A*	3 / 3	62	45	45	0.30 ms

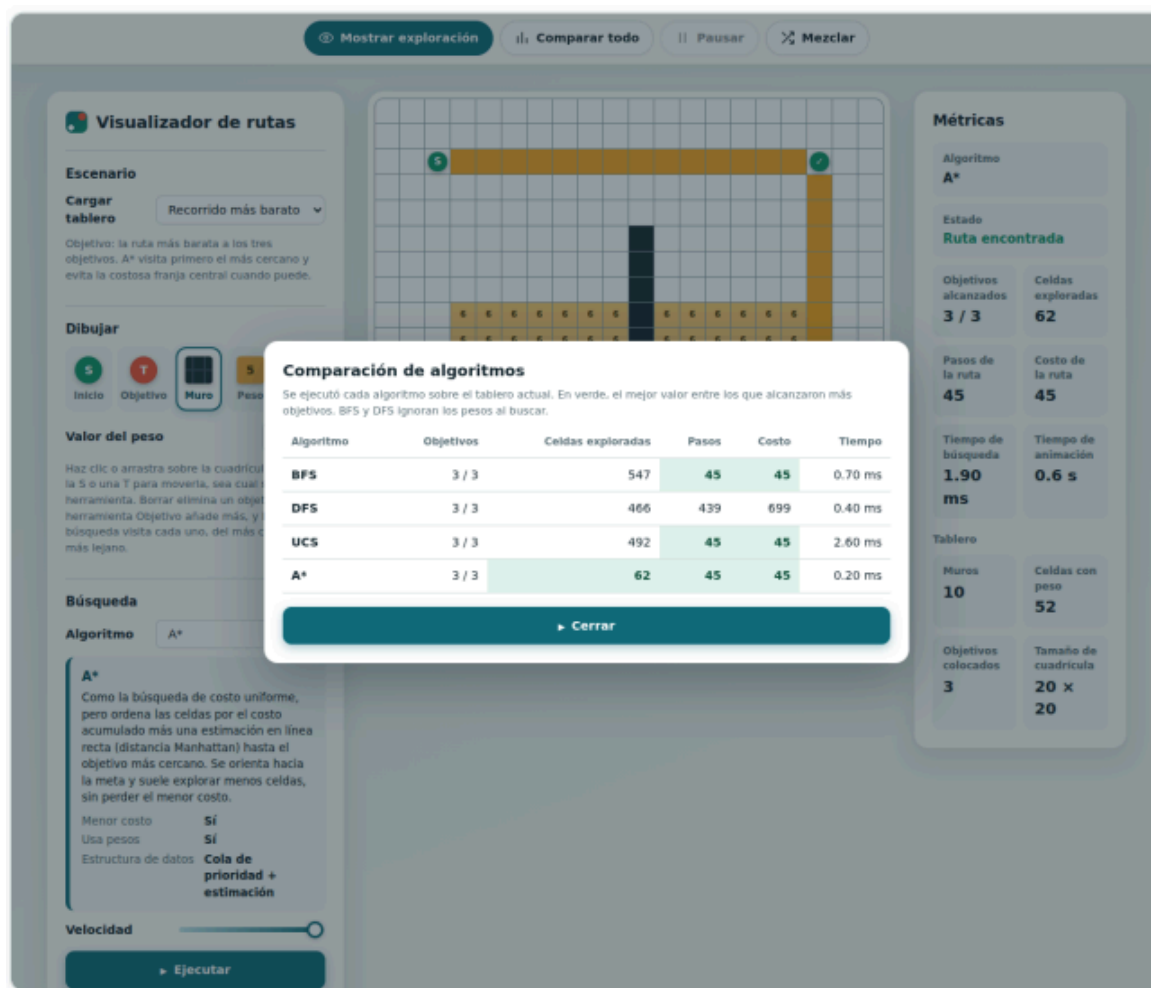


Fig. 4.5 — Cuadro de "Comparar todo" con los cuatro algoritmos sobre el mismo tablero.

6. Casos de Prueba Obligatorios

- **Ausencia de inicio/meta:** El sistema impide iniciar la ejecución y alerta al usuario si falta el nodo S o T.
- **Varias metas:** El algoritmo procesa la lista de objetivos, trazando rutas consecutivas desde el inicio hacia el primer objetivo más viable y así sucesivamente.
- **Pesos y Obstáculos:** Los muros actúan como valores nulos absolutos. Los pesos se suman correctamente en UCS y A*.
- **Edición post-ejecución:** Al dibujar sobre una ruta terminada, el mapa se limpia automáticamente para evitar solapamientos gráficos.
- **UCS vs A* en pesos:** He comprobado en los mapas de alto coste que AMBOS algoritmos devuelven exactamente el mismo coste total óptimo, validando que la heurística Manhattan en A* es admisible.

Comparación de algoritmos

Se ejecutó cada algoritmo sobre el tablero actual. En verde, el mejor valor entre los que alcanzaron más objetivos. BFS y DFS ignoran los pesos al buscar.

Algoritmo	Objetivos	Celdas exploradas	Pasos	Costo	Tiempo
BFS	3 / 3	683	63	115	0.50 ms
DFS	3 / 3	432	177	193	0.10 ms
UCS	3 / 3	704	69	73	0.20 ms
A*	3 / 3	301	69	73	0.30 ms

► Cerrar

7. Registro de Uso de Asistentes IA

Para esta práctica he utilizado Claude (Anthropic) como asistente de programación en su interfaz web. Siguiendo los criterios de la rúbrica, mi trabajo no consistió en copiar a ciegas, sino en dirigir el desarrollo de forma iterativa y verificar que todo funcionara:

- **Consultas principales que le hice:**
 - En primer lugar, le solicité generar la arquitectura base de la aplicación web estructurada en tres archivos independientes (HTML, CSS y JS), configurando el tablero de 20x20 e implementando la lógica inicial para los algoritmos BFS, DFS, UCS y A*, con soporte para inicio, metas múltiples, muros y pesos.
 - Posteriormente, le pedí incorporar interactividad mediante eventos de puntero para permitir el reposicionamiento de los nodos de inicio y meta mediante arrastre directo sobre la cuadrícula.
 - Tras detectar un conflicto de interacción en el navegador, le indiqué ajustar la prioridad de eventos para evitar que la herramienta activa (como pintar muros) sobrescribiera los nodos cuando la intención era arrastrarlos.
 - Le solicité fijar las dimensiones de la cuadrícula a un tamaño constante en píxeles (616x616 px) para evitar deformaciones según la resolución de la pantalla, además de diseñar e integrar el panel lateral de métricas para monitorizar los resultados en tiempo real.
- **Qué acepté y qué supervisé:** Acepté la estructura de código propuesta, pero me encargué de probar cada botón y cada algoritmo directamente en el navegador. De hecho, durante las pruebas detecté que las descripciones de dos escenarios estaban intercambiadas y una a medio traducir en el código, por lo que le pedí que lo corrigiera.
- **Cómo comprobé que funcionaba:** Comprobé que no hubiera fallos en la consola del navegador, ejecuté los cuatro laberintos paso a paso confirmando que A* y UCS daban el mismo coste óptimo, y verifiqué que al quedar encerrado el algoritmo terminará en 'Sin ruta' en vez de congelar la página.